PowerFlowNet: Leveraging Message Passing GNNs for Improved Power Flow Approximation

Nan Lin *Student Member, IEEE*, Stavros Orfanoudakis *Student Member, IEEE*, Nathan Ordonez Cardenas, Juan S. Giraldo, *Member, IEEE*, Pedro P. Vergara, *Senior Member, IEEE*

arXiv:2311.03415v1 [cs.LG] 6 Nov 2023

***Abstract*—Accurate and efficient power flow (PF) analysis is crucial in modern electrical networks' efficient operation and planning. Therefore, there is a need for scalable algorithms capable of handling large-scale power networks that can provide accurate and fast solutions. Graph Neural Networks (GNNs) have emerged as a promising approach for enhancing the speed of PF approximations by leveraging their ability to capture distinctive features from the underlying power network graph. In this study, we introduce PowerFlowNet, a novel GNN architecture for PF approximation that showcases similar performance with the traditional Newton-Raphson method but achieves it 4 times faster in the simple IEEE 14-bus system and 145 times faster in the realistic case of the French high voltage network (6470rte). Meanwhile, it significantly outperforms other traditional approximation methods, such as the DC relaxation method, in terms of performance and execution time; therefore, making PowerFlowNet a highly promising solution for real-world PF analysis. Furthermore, we verify the efficacy of our approach by conducting an in-depth experimental evaluation, thoroughly examining the performance, scalability, interpretability, and architectural dependability of PowerFlowNet. The evaluation provides insights into the behavior and potential applications of GNNs in power system analysis.**

***Index Terms*—Power Flow State Estimation, Deep Learning, Graph Neural Network.**

I. Introduction

THE complexity of electrical power systems is continuously rising, largely attributed to the substantial integration of decentralized renewable energy resources. Within this context, power flow (PF) stands as a fundamental challenge in ensuring the stability of power systems, playing a pivotal role in both the operational management and long-term planning of electrical networks. At its core, PF is a mathematical problem that revolves around determining the voltages at various buses, a task accomplished by solving a set of nonlinear equations, which are inherently linked to the network configuration, load distribution, and generation characteristics [1]. Traditional methods, such as Newton-Raphson [2], the Gauss-Seidel [3], and the fast-decoupled methods, have excellent accuracy and convergence properties but scale slowly with larger power systems, particularly in long-term planning of national grids with thousands of buses. In contrast, the DC power flow (DCPF) technique [4] simplifies this problem into a linear one by making assumptions about the voltage magnitude, but at the cost of accuracy. Consequently, there is a pressing requirement for innovative algorithms capable of efficiently solving the PF for significantly larger networks. These novel algorithms should aim to balance the need for speed without compromising accuracy, ultimately facilitating the streamlined operation and planning of large real-world electrical grids.

Modernizing the grids with the addition of accurate smart metering and data acquisition systems has enabled the development of Machine Learning (ML) methods for accurate and efficient power system analysis [5]. ML PF methods use historical operation data gathered from the metering infrastructure and try to approximate the power system state based on them. In [6], the authors effectively transformed the nonlinear PF relationship into a linear mapping within a higher-dimensional state space, resulting in a substantial improvement in the precision of the calculation process. Similarly, Chen et al. [7] mitigate the errors of model-based PF linearization approaches by approximating the nonlinear PF equations in a data-driven manner. For instance, in [8], the data-driven PF method comprises two distinct stages: offline learning and online computing. During the offline learning stage, a learning model is developed utilizing the proposed exact linear regression equations, which is subsequently solved by applying the ridge regression method to mitigate the impact of data collinearity. Subsequently, the need for nonlinear iterative calculations is obviated in the online computing stage, thereby streamlining the computation process. Furthermore, Liu et al. [9] reformulated PF into a regression model, leveraging the structural attributes of AC power flow equations, including Jacobian matrix-guided constraints, to substantially reduce the search space. Overall, all these simple ML methods have outperformed the DCPF method [10]; however, the scalability issues still do not allow for the application of these methods in large power systems.

Numerous deep learning techniques have been developed, promising to find more scalable solutions. Many studies emphasize the use of physics-informed methodologies that leverage the inherent characteristics of the problem. For example, Hu et al. [11] incorporate an auxiliary task for PF model

This work used the Dutch national e-infrastructure with the support of the SURF Cooperative using grant no. EINF-6262. This publication is part of the project ALIGN4energy (with project number NWA.1389.20.251) of the research programme NWA ORC 2020 which is (partly) financed by the Dutch Research Council (NWO). Stavros is by the HORIZON Europe Drive2X Project 101056934.

Nan Lin, Stavros Orfanoudakis, and Pedro P. Vergara are with the Intelligent Electrical Power Grids (IEPG) Section, Delft University of Technology, Delft, The Netherlands (emails: n.lin, s.orfanoudakis, p.p.vergarabarrios@tudelft.nl).

Nathan Ordonez Cardenas is with the Computer Science Department, Delft University of Technology, Delft, The Netherlands (email: n.a.ordonezcardenas@student.tudelft.nl).

Juan S. Giraldo is with the Energy Transition Studies Group, Netherlands Organization for Applied Scientific Research (TNO), Amsterdam, The Netherlands (email: juan.giraldo@tno.nl)

The first two authors contributed equally.

reconstruction, wherein the neural network (NN) based PF solver is effectively regularized by encoding varying levels of Kirchhoff's laws and system topology into the reconstructed PF model, consequently ensuring adherence to physical laws and constraints. In [12], the training process of the NN is enhanced through a training process guided by physical PF equations, such as incorporating branch flows as a penalty term in the NN's objective function and simplifying the backpropagation update gradients based on transmission grid's physical characteristics to expedite training without compromising effective model guidance. Similarly, Li et al. [13] use NNs to estimate the distribution system state equations by creating virtual nodes to represent nodes without any smart metering. An alternative approach is to enhance a classical solver (Gauss-Newton) by training an NN to serve as a regularizer [14]. However, it is important to note that classic deep learning approaches lack permutation equivariance, leading to limited generalization capabilities and reduced efficiency in real-world scenarios.

GNNs [15] have garnered significant attention in recent years due to their capacity to leverage graph-structured data by aggregating information from neighboring nodes while also being permutation equivariant, i.e., allowing the same NN to be used for problems with a different number of nodes. Consequently, researchers have explored diverse GNN architectures to tackle the PF problem, given that the grid topology often exhibits a sparsity pattern resembling a graph with useful connectivity information. In particular, GNNs have been successful in various tasks related to power systems [16], such as fault scenario application, time-series prediction, power flow calculation, and data generation. Wu et al. [17] utilized graph convolution networks (GCN) and recurrent NNs inspired by the physics of power systems to derive the PF equations. In [18], a weakly supervised learning approach is employed based on the PF equations that do not require labeled data, however, at the cost of accuracy compared to fully supervised methods. Furthermore, Pagnier et al. [19] improved some improvements on the previous method, such as implementing a specific encoder-decoder loss for the prediction of effective PF model parameters. Meanwhile, [20] improves upon an iterative-based GNN approach by replacing the pre-computed loss values with a purely physics-constraint loss. Despite the proliferation of various studies in this domain, little attention has been directed toward investigating the effectiveness of proposed methodologies for large power networks. Consequently, a comprehensive evaluation and comparison of ML and GNN-based approaches concerning accuracy, computational complexity, scalability, and interpretability are imperative to discern the most promising techniques for PF analysis.

To address the scalability and other aforementioned real-world application concerns, we propose PowerFlowNet. PowerFlowNet is a novel GNN architecture specifically designed to leverage electrical power networks' structural characteristics and interconnectedness, enabling efficient approximation of the PF. In practice, PowerFlowNet transforms the PF into a GNN node-regression problem by representing each bus as a node and each transmission line as an edge while maintaining the network's connectivity. The key advantage of PowerFlowNet lies in its ability to swiftly and accurately predict the complete PF, regardless of the network size; thus, enabling it to be applied for realistic power grid planning and operation processes. Specifically, our thorough experimental evaluation demonstrates PowerFlowNet's accuracy and execution speed compared to traditional methods, such as DCPF, while its performance is similar to the most trusted Newton-Raphson method but with significantly lower execution time. The much lower execution time will be extremely useful in particular scenarios where it is required to estimate thousands of PF in a short duration, given the high uncertainty introduced by distributed energy resources. Additionally, an in-depth analysis of the proposed approach's scalability, architectural dependability, and interpretability is conducted, aiming to provide valuable insights into the broader applicability of GNN algorithms in the power system domain.

In detail, the key contribution of this paper is summarized as follows:

- We introduce a novel GNN architecture for high-quality PF approximation with up to two orders of magnitude speedup compared to the Newton-Raphson method. Its distinctiveness, compared to existing PF GNN approaches, lies in its adept utilization of the capabilities from message passing GNNs [21] and high-order GCN [22], in a unique arrangement called PowerFlowConv, for handling a trainable masked embedding of the network graph. This innovative approach renders the proposed GNN architecture remarkably scalable, presenting an effective solution for the PF problem.

II. GNNs and Classic PF Algorithms

This section provides an overview of the fundamental background of GNNs and the methodologies they employ. Furthermore, it delves into the traditional techniques used in PF analysis, including the Newton-Raphson and DCPF methods.

A. Graph Neural Networks

GNNs are a class of ML models designed to operate on graph-structured data, which can represent complex relationships and interactions between entities. GNNs have gained significant attention due to their ability to capture and leverage the structural information inherent in graphs. In detail, a graph $\mathcal{G}(\mathcal{N}, \mathcal{E})$ can be defined as a set of $\mathcal{N}$ nodes and $\mathcal{E}$ edges with node features $\boldsymbol{x}_i \in \mathbb{R}^F$ and edge features $\boldsymbol{x}^e_{i,j} \in \mathbb{R}^{F_e}$, where $i \in \mathcal{N}$ and the pair $(i, j) \in \mathcal{E}$. More specifically, GNNs use parts of regular NNs to help them understand and work with the structure of these graphs.

1) Message Passing NNs: At the heart of GNNs lies the concept of message passing [21]. Message passing allows information to propagate through the graph by iteratively aggregating and updating node features based on their neighborhood relationships. In each message passing step, each node receives information from its neighbors, performs local computations, and then transmits updated information to its neighbors (Fig. 1). Specifically, the message passing phase runs for T timesteps for every node with hidden state $\boldsymbol{h}^t_i$

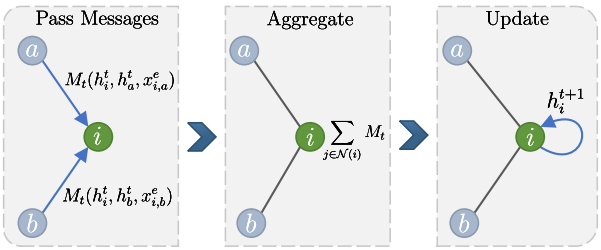


Fig. 1. A message passing step of node i consisting of message creation, aggregation, and update of the hidden state.

that is updated based on the message function $M_t(\cdot)$ and the node update function $U_t(\cdot)$. Therefore, for each node i with neighbors $\mathcal{N}(i)$, each iterative message update $\boldsymbol{v}_i^{t+1}$ is

$$\boldsymbol{v}_i^{t+1} = \sum_{j\in\mathcal{N}(i)} M_t(\boldsymbol{h}_i^t, \boldsymbol{h}_j^t, \boldsymbol{x}_{i,j}^e), \tag{1}$$

$$\boldsymbol{h}_i^{t+1} = U_t(\boldsymbol{h}_i^t, \boldsymbol{v}_i^{t+1}). \tag{2}$$

As shown in (1), the edge features $\boldsymbol{x}_{i,j}^e$ can also be included in the message creation step. After the message passing phase ends, the final feature vector for the whole graph is denoted as the output of a function $R(\cdot)$

$$\hat{\boldsymbol{y}}(\mathcal{G}) = R(\{\boldsymbol{h}_i^{\mathsf{T}} | i \in \mathcal{G}\}). \tag{3}$$

Notice that the functions $M_t(\cdot), U_t(\cdot)$, and $R(\cdot)$ are designed by the user and should be fully differentiable so that they can be trained and used by graph ML tasks. In addition, $R(\cdot)$ is often designed to be invariant to permutations of the set of node states to satisfy the equivariant feature of GNNs. Overall, message passing enables all nodes to gather and integrate information from their local context, incorporating both the features of neighboring nodes and the graph structure.

2) Graph Convolutional Networks: One other popular type of GNN is the GCNs [15], which introduces convolution operations on graphs. Similar to convolution neural networks (CNNs) for regular grid-like data, GCNs apply filters to node features and aggregate information from neighboring nodes. By leveraging the local connectivity of the graph, GCNs can capture both node-level patterns and the information of the K-hop neighbors. In particular, a graph convolutional layer of order K is defined as

$$\hat{\boldsymbol{y}}(S, X) = \sigma\left(\sum_{k=0}^{K-1} S^k X W\right). \tag{4}$$

Here, a GCN layer is a function of the graph shift operator $S \in \mathbb{R}^{N\times N}$, node features $X \in \mathbb{R}^{N\times F}$, and trainable weight matrix $W \in \mathbb{R}^{F\times F}$. Matrix S can be the adjacency matrix A, or a more complex representation such as the Laplacian transformation L. Moreover, σ represents a non-linear function, such as rectified linear units (ReLU), which is employed to enable the approximation of every target non-linear function. The parameter K holds significance as it dictates the K-hop[1] neighboring nodes are taken into account during each node convolution. Consequently, a larger value of K entails the inclusion of more nodes.

[1]Nodes with distance K from the original node.

3) Topology Adaptive GCN: Many variants of the GCN layer have been proposed with one of them being the topology adaptive graph convolution operator (TAGConv) [22]. TAGConv addresses a limitation of traditional GCN by adaptively learning the importance of different neighboring nodes during message passing. TAGConv assigns weights to the neighbors based on their relevance to the target node. Specifically, a TAGConv layer of order K is defined as

$$\hat{\boldsymbol{y}}(S, X) = \sigma\left(\sum_{k=0}^{K-1} S^k X W_k\right). \tag{5}$$

Here, the trainable weight matrix $W_k \in \mathbb{R}^{F\times F}$ is different for every K. This adaptivity enables TAGConv to effectively capture the local structure and important dependencies in the graph, resulting in improved performance in various tasks.

B. Classic Power Flow Analysis Methods

The PF problem is mathematically formulated as solving a set of non-linear equations. A power system has a set of buses (nodes) $\mathcal{N}$ and a set of transmission lines (undirected edges) $\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N}$. For each node $i \in \mathcal{N}$, the PF equations are derived from Kirchhoff's current law and are expressed in complex form as

$$P_i^{\mathrm{L}} - P^{\mathrm{G}} + j(Q_i^{\mathrm{L}} - Q^{\mathrm{G}}) = \sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^* \tag{6}$$

where P_i and Q_i are the active and reactive power drawn from bus i to the ground, $\dot{V}_i = V_i^{\mathrm{m}}\angle\theta_i$ and $\dot{V}_j = V_j^{\mathrm{m}}\angle\theta_j$ are the voltage phasors at bus i and j, and $z_{ij} = r_{ij} + x_{ij}$ are the line impedance associated with edge $(i,j) \in \mathcal{E}$. At each node i, two of P_i, Q_i, V_i^{m}, θ_i are unknown. For a system with $|\mathcal{N}| = N$ nodes, there are $2N$ real-valued equations and $2N$ unknown real-valued variables. For simplicity, we denote $M = 2N$. The vector form is represented as

$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0} \tag{7}$$

where $\boldsymbol{x} \in \mathbb{R}^M$ containing every unknown variable at all nodes and $\boldsymbol{f} : \mathbb{R}^M \to \mathbb{R}^M$.

1) Newton-Raphson Algorithm: The Newton-Raphson algorithm [2] is an iterative method for solving non-linear equations. It starts with an initial guess $\boldsymbol{x}^{(0)} = (x_1^{(0)}, x_2^{(0)}, ..., x_M^{(0)})$ and at every step, it updates the solution with a correction term

$$\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \Delta\boldsymbol{x}^{(t)}. \tag{8}$$

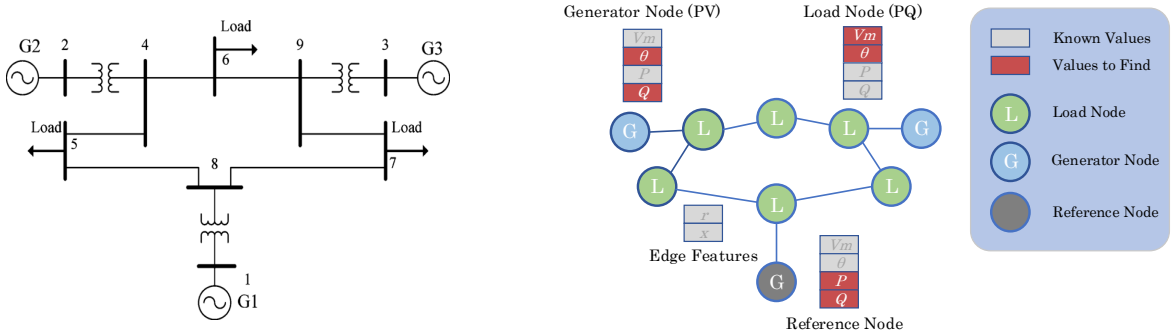
It calculates the update given $\boldsymbol{x}^{(t-1)}$ and a correction, assuming

$$\boldsymbol{f}(\boldsymbol{x}^{(t-1)} + \Delta\boldsymbol{x}^{(t)}) = \boldsymbol{0}. \tag{9}$$

With one order of Taylor approximation, we can get the following linear equations.

$$\boldsymbol{f}(\boldsymbol{x}^{(t-1)}) = -\mathbf{J}(\boldsymbol{f}(\boldsymbol{x}^{(\boldsymbol{t-1})})) \cdot \Delta\boldsymbol{x}^{(t)}. \tag{10}$$

Here, $\mathbf{J}(\boldsymbol{f}(\boldsymbol{x}^{(t-1)}))$ is the Jacobian matrix of $\boldsymbol{f}$ over $\boldsymbol{x}$ evaluated at $\boldsymbol{x}^{(t-1)}$. In every step, we need to solve the above equations once, involving an $M \times M$ matrix inversion. Newton-Raphson method usually converges very fast in the power flow problem, but because of the inversion, the computation time grows quickly with the number of nodes [23].



(a) Standard IEEE 9-case. (b) GNN equivalent of the IEEE 9-case.

Fig. 2. Interpreting the PF problem of the IEEE 9-case into a GNN node regression problem. In detail, Fig. 2a shows the standard IEEE 9-case, and Fig. 2b illustrates how each bus is transformed into a load, generator, or reference node while keeping the same line connectivity. Each node has different known and unknown features $\boldsymbol{x} = (V^m, \theta, P, Q)$ depending on its type. The ultimate goal of this GNN problem is to approximate all the features for all nodes, thus solving the PF problem.

2) DC Power Flow Approximation: DCPF is an approximation of the ACPF. In DCPF, the voltage is assumed constant at all buses; therefore, the voltage and reactive power differences are 0, so the ΔV and ΔQ terms are neglected [4]. Subsequently, the PF equations can be further simplified to a linear problem that does not require an iterative solution. In detail, the relation between the active power (P_i) and the voltage angle (θ_i) of each bus is

$$P_i = \Re\left[\sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^*\right] \approx \sum_{j\in\mathcal{N}(i)} \frac{\theta_j - \theta_i}{x_{i,j}}. \tag{11}$$

III. PowerFlowNet Framework

This section presents a thorough explanation of PowerFlowNet, accompanied by our interpretation of PF formulation in the domain of GNN algorithms.

A. Interpreting Power Flow as a GNN Problem

The primary objective of a PF study is to determine the voltage magnitudes and angles for a given load, generation, and network state, facilitating subsequent calculations of line flows and losses. This problem lends itself seamlessly to a GNN approach, as the electrical network topology can be represented as an undirected graph $\mathcal{G}(\mathcal{N}, \mathcal{E})$, where the set of nodes $\mathcal{N}$ corresponds to all the buses and the set of edges $\mathcal{E}$ represents power lines. The node features, denoted as $X \in \mathbb{R}^{N\times F}$, encompass various attributes for each bus, including voltage magnitude (V_i^{m}), voltage angle (θ_i), active power (P_i), and reactive power (Q_i). Similarly, the edge features, denoted as $X^e \in \mathbb{R}^{E\times F_e}$, encompass the resistance (r_{ij}) and reactance (x_{ij}) attributes for each line $(i,j) \in \mathcal{E}$.

Illustrated in Fig. 2, the conversion of a transmission network into a graph representation involves mapping the buses to nodes while preserving the interconnectedness of the lines. The nodes are categorized into three types: a) load (PQ) nodes where the P_i and Q_i are known, b) generator (PV) nodes where V_i^{m} and P_i are known, and c) reference nodes where both V_i^{m} and θ_i are known. The edge features X^e are assumed to be given. The objective of this task is to predict the missing node features for every node $i \in \mathcal{N}$, thereby rendering this problem as an intriguing node regression task.

B. PowerFlowNet Architecture

PowerFlowNet is a GNN approach that reconstructs every node's full feature vector $\hat{\boldsymbol{x}}_i = (V_i^{\text{m}}, \theta_i, P_i, Q_i)$ given partial information of the problem, such as the adjacency matrix of the graph A, the known node features $X = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]^\mathsf{T}$, and the edge features $X^e = [\boldsymbol{x}^e{}_{0,0}, \ldots, \boldsymbol{x}^e{}_{i,j}]^\mathsf{T}$. Our proposed model consists of a mask encoder and a stack of our novel Power Flow Convolutional layers (PowerFlowConv). Initially, the mask encoding layers transform the input features into a continuous embedding using a feature mask as a parameter. Then, as illustrated in Fig. 3, the encoded graph features are fed to the stack of Power Flow Convolutional operations consisting of a unique arrangement of message passing and TAGConv layers. This way, information from every node and edge is aggregated so that complete feature matrix $\hat{X} = [\hat{\boldsymbol{x}}_1, \ldots, \hat{\boldsymbol{x}}_N]^\mathsf{T}$ is predicted. A detailed explanation of PowerFlowNet's components and procedures is presented next.

1) Mask Encoder: In the PF problem, each node has different known and unknown features. The goal is to predict the unknown features while keeping the known ones unchanged. This means that our NN should know which features must be predicted. Consequently, for every input node with feature vector $\boldsymbol{x}_i$ we create a binary mask $\boldsymbol{m}_i \in \mathbb{R}^F$ where 0 represents the known and 1 the unknown features. For example, the mask of a load (PQ) node with a feature vector $\boldsymbol{x}_i = (V_i^m, \theta_i, P_i, Q_i)$ and unknown values V^m and θ, will be $\boldsymbol{m}_i = (1, 1, 0, 0)$. Additionally, similar to the positional embeddings in Transformers [24], we propose using a mask encoder, which in practice is an MLP with one hidden layer that learns a continuous embedding for every binary mask. The exact mathematical operation for every $m_i \in M$, where $M \in \mathbb{R}^{N\times F}$, is

$$\hat{\boldsymbol{m}}_i = W_1\sigma(W_0\boldsymbol{m}_i + b_0) + b_1, \ \forall i \in \mathcal{N}, \tag{12}$$

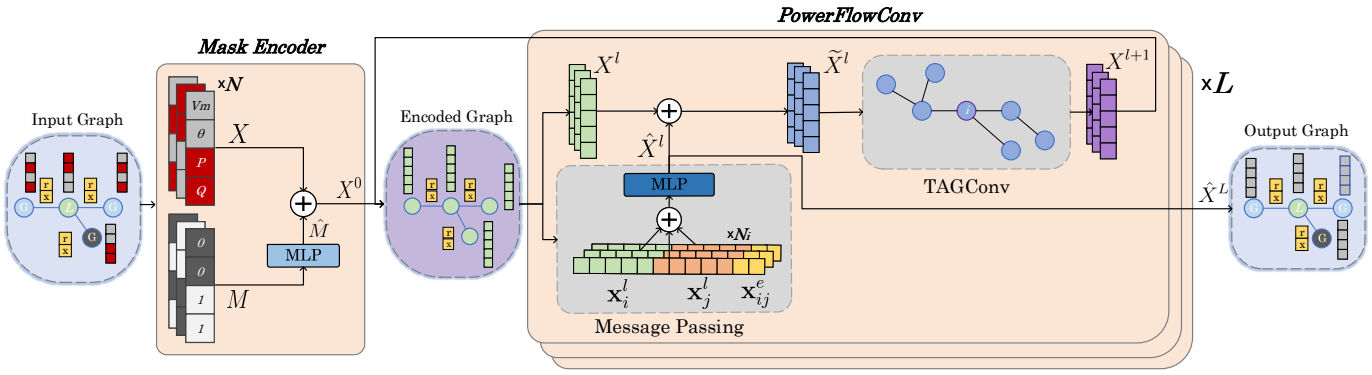


Fig. 3. The PowerFlowNet model architecture consists of a mask encoder and L PowerFlowConv layers. The input graph with incomplete feature information is fed node-by-node to the mask encoder to generate encoded graph features, where each node $n \in \mathcal{N}$ with $(\boldsymbol{x}_i, \boldsymbol{m}_i)$. Then, the encoded graph features are processed by a series of L sequential PowerFlowConv layers, each comprising a 1-step message passing and a high-order TAGConv. Finally, the complete output graph is produced.

where the function $\{0,1\}^F \to \mathbb{R}^F$, and the weight matrices W_0, W_1 and the biases b_0, b_1 are the basic trainable components of the Mask Encoder's MLP. Finally, to produce the encoded graph features X^l, the input node features X, and node mask embeddings are added up as

$$\boldsymbol{x}_i^0 = \boldsymbol{x}_i + \hat{\boldsymbol{m}}_i, \ \forall i \in \mathcal{N}. \tag{13}$$

2) Power Flow Convolutional Layer: The second part of our proposed architecture consists of L connected PowerFlowConv layers that sequentially process the encoded graph features and predict the final feature matrix $\hat{X}^L$, where $\hat{X}^L = [\hat{\boldsymbol{x}}_1^L, \ldots, \hat{\boldsymbol{x}}_N^L]^\mathsf{T}$ and $\hat{\boldsymbol{x}}_i^L = (V_i^\mathrm{m}, \theta_i, P_i, Q_i)$, $\forall i \in \mathcal{N}$. Each PowerFlowConv layer consists of an initial one-hop message passing step similar to [21] for extracting graph information, including the edge features and a K-hop TAGConv [22] on a higher-dimensional feature space. With the message passing step, we can effectively update our encoded graph features with information from the edge features, i.e., the line resistance and reactance, so that the TAGConv can efficiently exchange information in the K-hop neighborhood in one go while considering the edge features. Generally, graph convolutional layers, such as TAGConv do not use information from the edge features. Nevertheless, in the PF formulation, line characteristics play an important role. Therefore, by using message passing, we distill edge feature information into the node features that can later be used by TAGConv to provide high-quality PF approximations.

More specifically, the message passing equations $\forall i \in \mathcal{N}$ are described as

$$\hat{\boldsymbol{x}}_i^l = W_{MP_1}^l \cdot \sigma\left(W_{MP_0}^l \sum_{j \in \mathcal{N}_i} \langle \boldsymbol{x}_i^l, \boldsymbol{x}_j^l, \boldsymbol{x}_{i,j}^e \rangle + b_{MP_0}^l\right) + b_{MP_1}^l \tag{14}$$

where $\langle x_i^l, x_j^l, x_{i,j}^e \rangle$ is a concatenated vector of the described vectors fed to a two-layer MLP with an in-between ReLU activation function. Here again, the weight matrices and bias vectors $W_{MP_1}^l, b_{MP_1}^l, W_{MP_0}^l, b_{MP_0}^l$ are the trainable parameters of the MLP of layer l.

Afterward, the intermediate graph signals $\widetilde{X}^l$ of layer l are calculated as the sum of the encoded graph features X^l, and the processed message passing graph features $\hat{X}^l$, i.e. $\widetilde{X}^l = X^l + \hat{X}^l$, $\forall l \in \{0, \ldots, L\}$. Then, the signals $\widetilde{X}^l$ are processed by the TAGConv layer to generate the new encoded graph features X^{l+1} as in

$$X^{l+1} = \sigma\left(\sum_{k=0}^{K-1} (D^{-\frac{1}{2}} A D^{-\frac{1}{2}})^k \widetilde{X}^l W_k^l\right). \tag{15}$$

Here, $(D^{-1/2} A D^{-1/2})$ denotes the normalized adjacency matrix based on the diagonal degree matrix D. As shown in [22], the normalized adjacency matrix is often used in GNNs to provide computational stability guarantees. Then, the encoded graph features generated by the *l-th* PowerFlowConv layer are passed to the next layer until $l = L$. Finally, the message passing step is used in the last layer while the TAGConv step is discarded, as depicted in Fig. 3.

Notice that PowerFlowNet is equivariant to the permutation of the input nodes while also leveraging the edge features in every message passing step. Permutation equivariance refers to a property where the output of each node remains consistent regardless of the specific ordering of nodes in the input. This particular inductive bias arises naturally from the PF problem itself, as the PF equations (See Section II-B) also exhibit permutation equivariance. Additionally, we emphasize the importance of incorporating edge features into every step of our message passing process. This inclusion is crucial because edge features are fundamental variables within the PF equations, and their consideration is essential for making accurate predictions.

3) Loss Functions: The selection of the appropriate loss function is also important in generating high-quality PF approximations. Usually, purely physics-based losses are used in existing ML approaches for PF [18], [20]. However, using only physical losses is insufficient due to the peculiarities of the PF problem. Consequently, we conducted further experiments, which led us to utilize the Mean Squared Error (MSE) for the training of PowerFlowNet and develop the Masked L2 Loss as a better evaluation metric. In detail, MSE is defined as

$$\mathrm{MSE}\,(\boldsymbol{y}_i, \hat{\boldsymbol{x}}_i) = ||\boldsymbol{y}_i - \hat{\boldsymbol{x}}_i||_2^2 \tag{16}$$

where $\boldsymbol{y}_i$ represents the real values of the features and $\hat{\boldsymbol{x}}_i$ are the predicted values of node i.

To get a more precise value of the actual error in predicting only the unknown features, we developed the Masked L2 loss. This loss function is similar to MSE but only calculates the error for the unknown features,

$$\text{MaskedL2}\left(\boldsymbol{y}_i, \hat{\boldsymbol{x}}_i\right) = ||\boldsymbol{m}_i \circ (\boldsymbol{y}_i - \hat{\boldsymbol{x}}_i)||_2^2. \tag{17}$$

Here, the $\circ$ is the element-wise multiplication operator, and $\boldsymbol{m}_i$ is the binary mask of node i indexing only the features of interest, as defined in III-B1.

IV. Experimental Evaluation

This section presents the experimental setup employed in this study, followed by a comprehensive evaluation aimed at elucidating the strengths and weaknesses of the PowerFlowNet model. To gain deeper insights into the efficacy of PowerFlowNet[2] as a GNN approach for PF approximation, the conducted ablation studies focus on four distinct aspects of the proposed approach: performance, interpretability, scalability, and architectural dependability.

A. Experimental Setup

The evaluation of PowerFlowNet's performance requires the consideration of the varied attributes present in each power network, encompassing differences in size and topology. To address this requirement comprehensively, three distinct power network cases are evaluated, namely IEEE case-14, IEEE case-118, and case 6470rte [25], based on their contrasting characteristics. The case with 14 nodes represents a minimal grid topology characterized by a limited number of connections, while the 118 case exhibits a more intricate configuration with a significantly larger number of nodes. Moreover, the 6470rte case accurately emulates the scale and intricacy of the French very high voltage and high voltage transmission network. It is important to note that realistic large power grids such as the 6470rte case are rarely investigated in PF problems because of the inability of most methods to seamlessly scale up. In contrast, we evaluate PowerFlowNet in this realistic large-scale network to fully show its scalability potential.

1) Dataset Generation: All aforementioned cases were modeled in PandaPower [26] and solved using the Newton-Raphson method. For each case, a total of over 30.000 distinct graph samples were generated by perturbing the default case files. The generation of each sample involved sampling from random uniform or normal distributions. Specifically, the resistance (r in Ω) and reactance (x in Ω) values of each line were uniformly selected within 80% and 120% of their original values. In the case of generators, the voltage magnitude (V^m) was uniformly set within the range of $[1.00, 1.05]$ per unit, while the initial active powers (P_g in MW) were sampled from a normal distribution $\mathcal{N}(P_g, 0.1|P_g|)$, where the mean corresponds to the original active power and the standard deviation is 10% of it. Lastly, for buses with loads, their active power (P in MW) and reactive power (Q in MVAR) were randomly sampled from normal distributions $\mathcal{N}(P, 0.1|P|)$ and $\mathcal{N}(Q, 0.1|Q|)$, respectively. These samples were then partitioned, allocating 50% for training, 20% for validation, and 30% for testing purposes.

2) Training & Evaluation: To train the models, the AdamW optimizer [27] was employed, using a learning rate of 0.001, in conjunction with the OneCycleLR scheduler [28]. The training process encompassed up to 2000 epochs (for large models) and 1000 epochs (for medium and small networks), with each epoch comprising a batch size of 128 samples. In our standard implementation, we use 4 PowerFlowConv layers ($L = 4$) of order 3 ($K = 3$) to ensure that the computation subgraph of every node covers the majority of the input graph. Additionally, we use 128 nodes for the hidden dimension and a dropout rate of 0.2 at the end of each layer to reduce overfitting. We train the models with MSE loss as defined in Equation 16. We also experimented with training with the physical loss and the mixed loss, defined in Equations 20 and 21 respectively. These additional results can be found in Section IV-E.

a) Physical Loss: This is a node-wise physical loss [20], defined by the unbalanced power at each node. This is a self-supervised loss function since it does not require the actual values of every node.

$$\text{Physical}\left(\hat{X}\right) = \frac{1}{|\mathcal{N}|}\sum_{i\in\mathcal{N}} ||\Delta P_i||_2^2 + ||\Delta Q_i||_2^2 \tag{18}$$

$$\Delta P_i = \hat{P}_i - \Re\left[\sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^*\right] \tag{19}$$

$$\Delta Q_i = \hat{Q}_i - \Im\left[\sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^*\right] \tag{20}$$

This loss function is purely based on the PF equations where $\mathcal{N}(i)$ is the set of neighbor nodes of node i, $\dot{V}_i = \hat{V}_i \angle \hat{\theta}_i$, $(\hat{V}_i, \hat{\theta}_i, \hat{P}_i, \hat{Q}_i)$ is the i-th row of $\hat{X}$, and $r_{i,j}$ and $x_{i,j}$ refer to the resistance and reactance of line (i, j), respectively.

b) Mixed Loss: Finally, we propose the Mixed loss that combines the physical and MSE loss to efficiently incorporate the PF equations into PowerFlowNet. . In detail, the Mixed loss for the whole graph with predicted feature matrix $\hat{X}$ and actual real feature value matrix Y is defined as:

$$\text{Mix}\left(Y, \hat{X}\right) = w\cdot\text{MSE}\left(Y, \hat{X}\right) + \tau\cdot(1-w)\cdot\text{Physical}(\hat{X}) \tag{21}$$

where $w \in [0, 1]$ is a weight to balance the MSE and physical loss, and τ is a scaling factor bringing the physical loss to the same order of magnitude as the MSE loss.

c) Evaluation: During the evaluation, we compare the Masked L2 loss as a metric[3]. The execution times of each implemented method were examined using high-performance hardware configurations, including the "AMD RYZEN 7 5700X 8-Core" processor, "NVIDIA RTX 3060 TI 8GB"

[2]PowerFlowNet code, datasets, and trained models can be found at https://github.com/StavrosOrf/PoweFlowNet (will be public upon acceptance)*.

[3]We also experimented with training with Masked L2 loss, which resulted in almost the same performance as training with MSE loss. Therefore, we adopted the MSE loss because it gives a slightly better feature reconstruction.

TABLE I
PERFORMANCE COMPARISON OF POWERFLOWNET.

Case	14		118		6470rte	
Algorithms	Masked L2 Loss	Time (ms)	Masked L2 Loss	Time (ms)	Masked L2 Loss	Time (ms)
Newton-Raphson	≈ 0	17.0	≈ 0	20.0	≈ 0	580.0
DC Power Flow	45.74	8.0	99.87	10.0	510.5	30.0
Tikhonov Reg.	2.838	0.4	1.916	0.4	2.934	6100
3-Layer MLP	0.034	0.6	0.462	0.6	0.590	0.6
3-Layer GCN	0.239	1.0	1.257	1.0	1.957	2.0
PowerFlowNet	**0.002**	4.0	**0.022**	4.0	**0.303**	4.0

graphics card, and 32GB RAM, ensuring accurate and efficient measurements. Notably, it is important to emphasize the reproducibility of the experiments, as the availability of datasets, the PowerFlowNet framework, and the trained models provided ensure the feasibility of replicating the study's outcomes and procedures.

B. Performance Comparison

In this comparison study, PowerFlowNet is evaluated against other baseline methods for PF approximation across three distinct cases, as outlined in Table I. The performance assessment of each approach is conducted based on accuracy, quantified by employing the Masked L2 Loss, as well as the execution time on the same computing machine. The datasets used for evaluation were generated using the Newton-Raphson method, recognized for its accuracy in PF analysis, resulting in zero loss for these particular cases. As expected, the Newton-Raphson method has increased execution time as the scale of the PF problem grows. The second baseline method considered is the DCPF approach, which relaxes the nonlinear problem to a linear one, thereby serving as an upper-performance boundary for the comparison. However, the DCPF only calculates each bus' voltage angle and active power since it assumes the voltage magnitude to be constant across the grid. PowerFlowNet exhibits remarkable performance superiority in terms of loss, surpassing the DCPF method by several orders of magnitude, depending on the case. Additionally, our proposed approach demonstrates significantly reduced execution time compared to the Newton-Raphson method, with speed improvements of $4\times$ in the 14-node case, $5\times$ in the 118-node case, and an impressive $145\times$ in the 6470rte case. Notably, owing to the GNN architecture of PowerFlowNet, the execution time remains consistently low across different cases. This inherent scalability characteristic of GNNs represents a significant advantage over traditional methods, enabling the real-time approximation of PF in extensive power networks. Note that the computation time of Newton-Raphson is acceptable (≈ 0.6s) when computing a small number of cases but might not be enough when thousands of computations are needed in tasks such as planning where the uncertainty of distributed energy resources is introduced. For example, the Newton-Raphson method would take around 20 minutes to calculate 2000 PF cases for the 6470rte grid, while PowerFlowNet would only need around 1.3 minutes.

Furthermore, a comprehensive evaluation of alternative traditional ML approaches was conducted to ascertain the unique benefits of utilizing PowerFlowNet. Initially, the Tikhonov regularizer [29], a method designed to enforce smoothness while considering the underlying graph structure, was assessed. However, as indicated in Table I, its performance consistently fell behind the other ML methods in terms of loss across all cases, as well as in execution time within the context of large-scale power networks. Subsequently, a three-layer Multilayer Perceptron and a simplified GNN consisting of merely three layers of GCNs [15] were evaluated. Although neither approach surpassed PowerFlowNet in terms of loss (MLP was $16\times$ worse and the GCN $12\times$ worse in the 14-case), it is noteworthy that the MLP exhibited a substantially faster execution time, differing by an order of magnitude ($\approx 6.5\times$ faster).

TABLE II
AVERAGE ABSOLUTE DENORMALIZED ERROR

Case	V^m (p.u.)	θ (*deg*)	P (MW)	Q (Mvar)
14	0.0008 ± 0.0006	0.09 ± 0.07	0.02 ± 0.03	0.13 ± 0.16
118	0.0022 ± 0.0018	0.64 ± 0.54	0.18 ± 0.36	1.20 ± 1.47
6470rte	0.0072 ± 0.0061	10.2 ± 7.89	0.23 ± 0.42	0.12 ± 6.46

C. Denormalized Error Distributions

After showcasing the performance of PowerFlowNet in three different power networks, each with unique characteristics, it is important to delve deeper into the results and visualize the denormalized error distributions. This analytical approach allows us to gain valuable insights into the areas where more accurate predictions can be achieved, as well as the conditions under which these improvements are most evident. Such insights hold practical significance for power systems operators, enabling them to leverage PowerFlowNet effectively in real-world scenarios involving network operation and strategic planning. Table II shows the average error distribution for every one of the four predicted values[4].

Notice that the proposed model achieves precise predictions of p.u. voltage magnitude for all cases, exhibiting an average error of 8×10^{-4} p.u. in the 14-node scenario and 72×10^{-4} in the notably more intricate 6470rte case, all while maintaining a consistently low standard deviation across each case. Additionally, the average absolute error remains minimal for both active and reactive power with respect to the network's actual demand and load, with an actual error of 20 kW for active power and 130 kvar for reactive power in the best-case scenario. Nevertheless, it is important to note a significant rise in the standard deviation of predicted reactive power error, particularly in the 6470rte case, where it reaches as high as 6460 kvar. Similarly, the voltage angle prediction error is barely noticeable in the 14-node case, being only 0.09 degrees, but increases significantly to 10.2 degrees in the 6470rte case. This occurs because the range of the voltage angles across larger networks network, i.e., the 6470rte case, is much greater.

Furthermore, we can observe the per-node denormalized error distribution for every case in Fig. 4. Notice that the

[4]The error distributions include only the predictions of the unknown values.

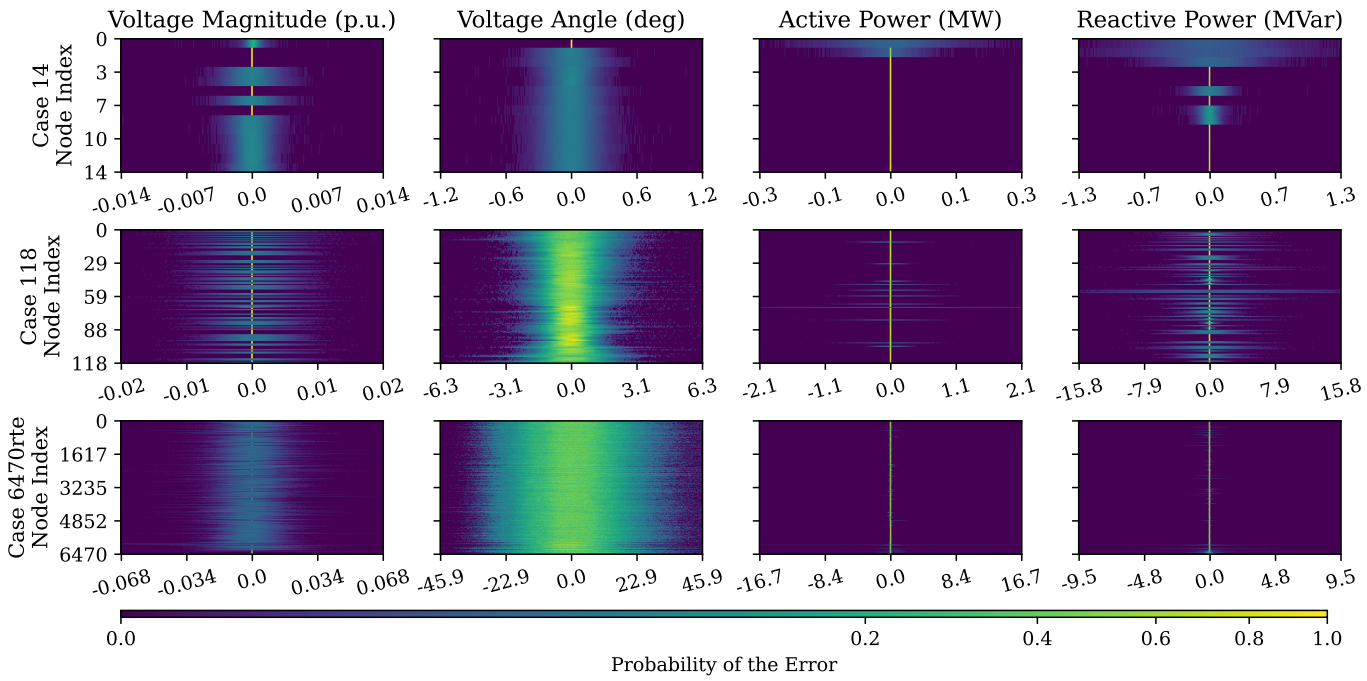


Fig. 4. Probability density function of the actual not-normalized error for every node in the test dataset. The brighter the color, the highest the probability of the prediction error of PowerFlowNet being in that region.

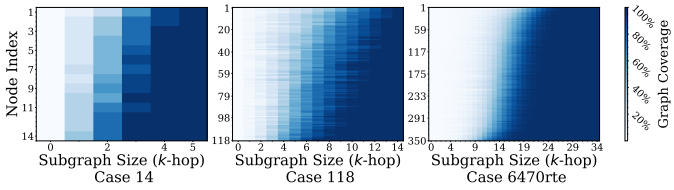


Fig. 5. Illustration of graph coverage defined as the number of nodes included in a subgraph, plotted per K-hop size and sorted from slowest to fastest growth for every node.

prediction error for nodes where we already know the value is practically zero (indicated by the vertical yellow line). Although the average voltage angle error is higher for the 6470rte case (10.2 degrees as shown in Table II), the average voltage magnitude error is in the same range (less than 0.008 p.u.) as in the smaller networks, ensuring that the developed PowerFlowNet is accurate enough for fast checking of operational constraints for large numbers of scenarios regardless of network size.

D. Interpretability Study

To gain insights into the locality of information used by PowerFlowNet to make predictions, we aimed to investigate potential affordances in using subgraphs as a more scalable and generalizable approach to tackle the challenges posed by the complex PF prediction task.

We examined K-hop subgraphs across all three cases, where, for reference, a PowerFlowNet model with a kernel size of $K = 3$ and four layers of TAGConv ($L = 4$), a straightforward computation yields an effective receptive field spanning 12 nodes. In practice, the size of these receptive fields, measured in the number of nodes, will vary based on the specific graph topology. To better understand this topological effect, we looked at the speed at which K-hop subgraphs grow from each node in the graph, also defined as graph coverage. In detail, for each node, we made K-hop subgraphs for K between 1 and the graph diameter and measured the subgraph size after each hop, as shown in Fig. 5. This indicates the distribution of receptive field sizes seen during training; interestingly, it appears to form a Gaussian shape, particularly visible in the largest graph in the dataset with 6470 nodes. As expected the larger the graph size the more hopes are required to cover the whole graph. For example, 4 hops are required in the 14-node over 6 hops in the 118-node case, and 11 to 25 hops in the 6470rte case.

Afterward, those subgraphs were each passed through the PowerFlowNet model, and the Masked L2 loss was calculated only on the node from which the K-hop neighborhood was generated. This showed us the actual loss distribution among nodes depending on how far a K-hop neighborhood can reach. The detailed results are illustrated in Fig. 6. We noticed that this loss distribution has a different shape from the subgraph growth figure, and indeed there is variation in which nodes are more or less affected by a decrease in subgraph size.

Interestingly, we can observe that only around 3 hops are required to obtain the minimum loss for most of the nodes, experimentally proving the reason TAGConv of order $K = 3$ can achieve high-quality results. This also suggests that for most buses (nodes) in a power network, we could make PowerFlowNet predict the PF values even faster by only looking at smaller connected parts.

Finally, we plot the average loss across nodes in Fig. 7 and discover that neighborhood hop counts well below the effective

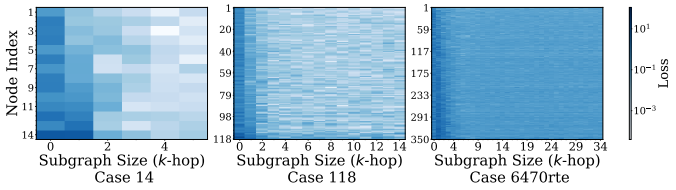


Fig. 6. Masked L2 loss on the central node of a k-hop subgraph for each node and k, sorted by the total node loss.

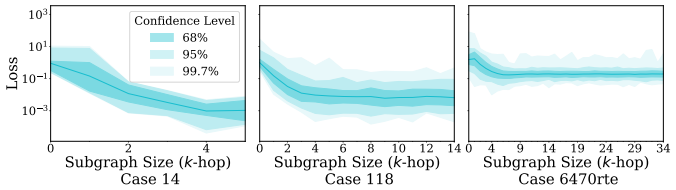


Fig. 7. Average Masked L2 loss on the central nodes of a k-hop subgraph for different numbers of k.

TABLE III
MASKED L2 LOSS OF SCALED MODELS ON THE 118 CASE.

Model	# Params	Trained with		
		MSE	Physical	Mixed
PowerFlowNet (Small)	32k	0.079	0.757	0.109
PowerFlowNet	357k	0.022	0.667	0.057
PowerFlowNet (Large)	7375k	**0.002**	**0.628**	**0.004**

receptive field's hop count are enough to make predictions practically identical to those when using the full graph. This indicates that local predictions in a large network are feasible. Most importantly, we argue that training on larger graphs could be made more efficiently by sampling subgraphs across different graphs available in the dataset, so as to make sure that stochastic gradient descent steps are performed using batches that capture more of the variation available in the dataset. Subsequently, this approach has the potential to augment the scalability aspects of PowerFlowNet.

E. Model Scalability Study

We demonstrate that our model has valuable scalability features; as we investigate how PowerFlowNet's performance scales with the model size, the size of the training dataset, and the loss function used.

Based on the standard setting PowerFlowNet (Medium) with $L = 4, h = 128$, we created two variants by modifying the total number of the model's trainable parameters by one order of magnitude as shown in Table III. In detail, we created PowerFlowNet (Small) with $L = 2, h = 64$ and PowerFlowNet (Large) with $L = 5, h = 512$. Table III shows the relation between the model size and the Masked L2 loss when trained with different loss functions for the 118-node case. We note that the performance scales very well with the model size. Most importantly, from the medium to large model, the Masked L2 loss is almost one order of magnitude lower, signifying the learning potential of PowerFlowNet.

The model performance does not differ greatly when trained with physical loss (as used in [20]). This means learning with physical loss is challenging regardless of the model's capability. However, we notice that the Masked L2 performance difference between training with MSE and mixed loss shrinks as we move to the large model. This suggests that this capability gain allows us to optimize both the MSE and the physical loss, thus making precise and physics-conforming predictions.

TABLE IV
PERFORMANCE EVALUATION FOR VARYING TRAINING CASES.

Eval. Case		118			14		
Eval. Metric		M. L2	MSE	Phys.	M. L2	MSE	Phys.
Train. Case	118+14 (L)	**0.0023**	**0.0011**	**4.9676**	**0.0014**	**0.0006**	**0.1336**
	118 (L)	0.0029	0.0014	5.4080	6.1998	2.5947	40.942
	118 (M)	0.0220	0.0105	58.9221	3.7860	3.7860	45.426

TABLE V
POWERFLOWNET COMPONENT SIGNIFICANCE ANALYSIS

Case	Full Model	Model 1-Layer	No Message Passing (MP)	1-Layer & No MP
14	**0.002**	0.11	0.78	0.08
118	**0.022**	0.38	1.01	0.24
6470rte	**0.303**	1.12	1.91	0.88

To further test PowerFlowNet's capability, we trained the large model on a mixed dataset composed of the 14 and the 118 cases. We unified the training loss function with MSE loss and evaluated the trained models using different metrics (MSE, Masked L2, and physical loss) on each of the two cases individually. The results are given in Table IV. After comparing with training large (L) and medium (M) models on the 118-node case, we noticed that even without explicitly bringing physical loss in training, the large models automatically learn the underlying physics, yielding more physics-conformed predictions. Large models are around $10\times$ better in terms of loss when trained and evaluated on the same case. When we augment the 118-node case training set with the 14-node case, the Large PowerFlowNet model performs even better than when trained only in the 118-node case (0.0006 less Masked L2 loss), achieving an excelling accuracy in the 14-node case (0.0006 less Masked L2 loss than just training in the 14 case). This suggests that the proposed large PowerFlowNet model can operate on multiple graphs and learn the underlying physics without sacrificing accuracy.

F. Architectural Ablation Study

Finally, architectural ablation experiments were carried out to demonstrate the vitality of PowerFlowNet's unique structures through component removal. Therefore, the experiments were designed to highlight how each PowerFlowNet component contributes to generating high-quality predictions. Table V depicts the results of these experiments. Table V is split into four variations of PowerFlowNet: the full model, PowerFlowNet with $L = 1$, PowerFlowNet without message passing, and PowerFlowNet with $L = 1$ and without message passing. Then, we can observe how the model performs in terms of Masked L2 loss in all different scenarios.

As observed, none of the PowerFlowNet's variations performed as well as the complete model. In the 1-layer case, it is visible that the model fails to capture distant node dependencies, resulting in up to $55\times$ higher error. Similarly, the model performs poorly (up to $390\times$ higher error) in the no message passing case since the edge features are completely ignored, highlighting the importance of incorporating the edge features. On the other hand, the 1-layer model without message passing performs the best (up to $40\times$) since its functionality is similar to stacked MLPs. Consequently, this analysis shows that every part of PowerFlowNet is important in achieving high-quality PF approximations.

V. Conclusions

In this paper, we presented PowerFlowNet, a novel data-driven algorithm that capitalized on the efficiency and capabilities of GNN operations applied to the power network's topology, leading to a precise approximation of the PF. Specifically, our approach transformed the traditional PF problem into a GNN node-regression task by representing buses as nodes and transmission lines as edges while preserving network connectivity. Our model featured a distinctive configuration involving a mask encoder in conjunction with a sequence of our proposed PowerFlowConv layers, designed to aggregate features across the entire graph and inherently learn the dynamics of the underlying PF. The results verified PowerFlowNet's ability to generate high-quality solutions for a diverse set of network cases including the 6470rte very large-scale power network of France. Most importantly, our method produces accurate PF predictions at a fraction of the time of traditional solvers, regardless of the scale of the network, outperforming the well-established Newton-Raphson method in terms of execution time while having closely comparable results. Ultimately, our ablation studies reveal the robust aspects and potential limitations of our approach, thereby enabling the versatile deployment of PowerFlowNet across a wide array of real-world power system operation and planning scenarios.

References

[1] M. Albadi, "Power flow analysis," in *Computational Models in Engineering*, K. Volkov, Ed. Rijeka: IntechOpen, 2019, ch. 5.

[2] V. M. da Costa, N. Martins, and J. L. R. Pereira, "Developments in the newton raphson power flow formulation based on current injections," *IEEE Trans. Power Syst.*, vol. 14, pp. 1320–1326, 1999.

[3] A. Eltamaly and A. A. Elghaffar, "Load flow analysis by gauss-seidel method; a survey," *International Journal of Mechatronics, Electrical and Computer Technology (IJMEC) 2305-0543, 2411-6173*, vol. x, 10 2017.

[4] H. Seifi and M. Sepasian, *Electric Power System Planning: Issues, Algorithms and Solutions*. Springer, 01 2011, vol. 49.

[5] Y. Liu, N. Zhang, Y. Wang, J. Yang, and C. Kang, "Data-driven power flow linearization: A regression approach," *IEEE Trans. on Smart Grid*, vol. 10, no. 3, pp. 2569–2580, 5 2019.

[6] L. Guo, Y. Zhang, X. Li, Z. Wang, Y. Liu, L. Bai, and C. Wang, "Data-Driven Power Flow Calculation Method: A Lifting Dimension Linear Regression Approach," *IEEE Trans. on Power Syst.*, vol. 37, no. 3, pp. 1798–1808, 5 2022.

[7] J. Chen, W. Wu, and L. A. Roald, "Data-driven piecewise linearization for distribution three-phase stochastic power flow," *IEEE Trans. on Smart Grid*, vol. 13, no. 2, pp. 1035–1048, 2022.

[8] Y. Chen, C. Wu, and J. Qi, "Data-driven Power Flow Method Based on Exact Linear Regression Equations," *Journal of Modern Power Systems and Clean Energy*, vol. 10, no. 3, pp. 800–804, 5 2022.

[9] Y. Liu, Y. Wang, N. Zhang, D. Lu, and C. Kang, "A data-driven approach to linearize power flow equations considering measurement noise," *IEEE Trans. on Smart Grid*, vol. 11, no. 3, pp. 2576–2587, 2020.

[10] S. Ramasamy, K. Ganesan, and V. Arunachalam, "Power Flow Parameter Estimation in Power System Using Machine Learning Techniques Under Varying Load Conditions," *International Journal of Electrical and Electronics Research*, vol. 10, no. 4, pp. 1299–1305, 2022.

[11] X. Hu, H. Hu, S. Verma, and Z.-L. Zhang, "Physics-guided deep neural networks for power flow analysis," *IEEE Trans. Power Syst.*, vol. 36, no. 3, pp. 2082–2092, 2021.

[12] Y. Yang, Z. Yang, J. Yu, B. Zhang, Y. Zhang, and H. Yu, "Fast Calculation of Probabilistic Power Flow: A Model-Based Deep Learning Approach," *IEEE Trans. on Smart Grid*, vol. 11, no. 3, pp. 2235–2244, 5 2020.

[13] H. Li, Y. Weng, V. Vittal, and E. Blasch, "Distribution grid topology and parameter estimation using deep-shallow neural network with physical consistency," *IEEE Trans. on Smart Grid*, pp. 1–1, 2023.

[14] Q. Yang, A. Sadeghi, G. Wang, G. B. Giannakis, and J. Sun, "Power system state estimation using gauss-newton unrolled neural networks with trainable priors," *2020 IEEE SmartGridComm*, pp. 1–6, 2020.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

[16] W. Liao, B. Bak-Jensen, J. R. Pillai, Y. Wang, and Y. Wang, "A review of graph neural networks and their applications in power systems," *J. Mod. Power. Syst. Clean Energy*, vol. 10, no. 2, pp. 345–360, 2022.

[17] T. Wu, I. L. Carreno, A. Scaglione, and D. Arnold, "Spatio-Temporal Graph Convolutional Neural Networks for Physics-Aware Grid Learning Algorithms," *IEEE Trans. on Smart Grid*, 2023.

[18] B. Habib, E. Isufi, W. v. Breda, A. Jongepier, and J. L. Cremer, "Deep statistical solver for distribution system state estimation," *IEEE Trans. Power Syst.*, pp. 1–12, 2023.

[19] L. Pagnier and M. Chertkov. Physics-Informed Graphical Neural Network for Parameter and State Estimations in Power Systems.

[20] B. Donon, R. Clément, B. Donnot, A. Marot, I. Guyon, and M. Schoenauer, "Neural networks for power flow: Graph neural solver," *Electric Power Systems Research*, vol. 189, p. 106547, 2020.

[21] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *International Conference on Machine Learning*. PMLR, Jul. 2017, pp. 1263–1272.

[22] J. Du, S. Zhang, G. Wu, J. M. Moura, and S. Kar, "Topology adaptive graph convolutional networks," *arXiv preprint arXiv:1710.10370*, 2017.

[23] V. M. da Costa, N. Martins, and J. L. R. Pereira, "Developments in the newton raphson power flow formulation based on current injections," *IEEE Trans. Power Syst.*, vol. 14, no. 4, pp. 1320–1326, 1999.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.

[25] C. Josz, S. Fliscounakis, J. Maeght, and P. Panciatici, "Ac power flow data in matpower and qcqp format: itesla, rte snapshots, and pegase," *arXiv: Optimization and Control*, 2016.

[26] L. Thurner, A. Scheidler, F. Schäfer, J. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun, "pandapower — an open-source python tool for convenient modeling, analysis, and optimization of electric power systems," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6510–6521, 11 2018.

[27] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2017.

[28] L. N. Smith and N. Topin, "Super-convergence: very fast training of neural networks using large learning rates," in *Defense + Commercial Sensing*, 2017.

[29] D. E. Hilt, D. W. Seegrist, U. S. F. Service., and N. F. E. Station, *Ridge, a computer program for calculating ridge regression estimates*. Upper Darby, Pa, Dept. of Agriculture, Forest Service, Northeastern Forest Experiment Station, 1977, vol. no.236.